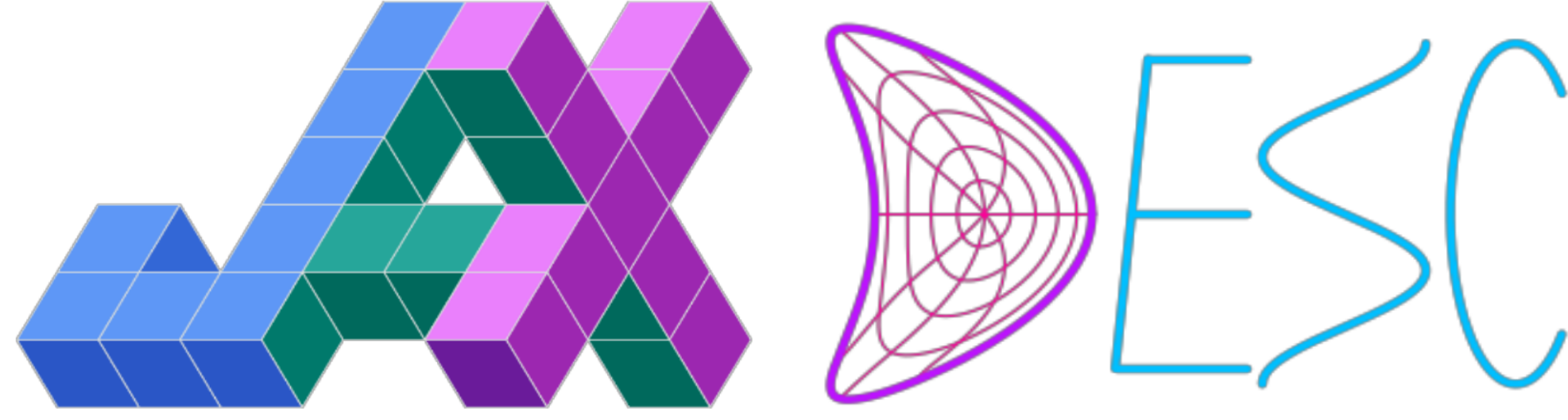


INTRODUCTION

DESC is an open-source stellarator equilibrium and optimization code which is written purely in Python and has an active developer team constantly making improvements, adding new functionality and resolves bugs that our users report.

DESC uses JAX heavily for the backend which gives us the ability to use automatic differentiation and jit compilation. Although, JAX gives us great strenght, it does also limit us sometimes. As JAX improves, and as we learn JAX tricks, DESC also gets faster and more efficient. Here, we will compare DESC version 0.10.4 and 0.14.0, the former is released a year ago.

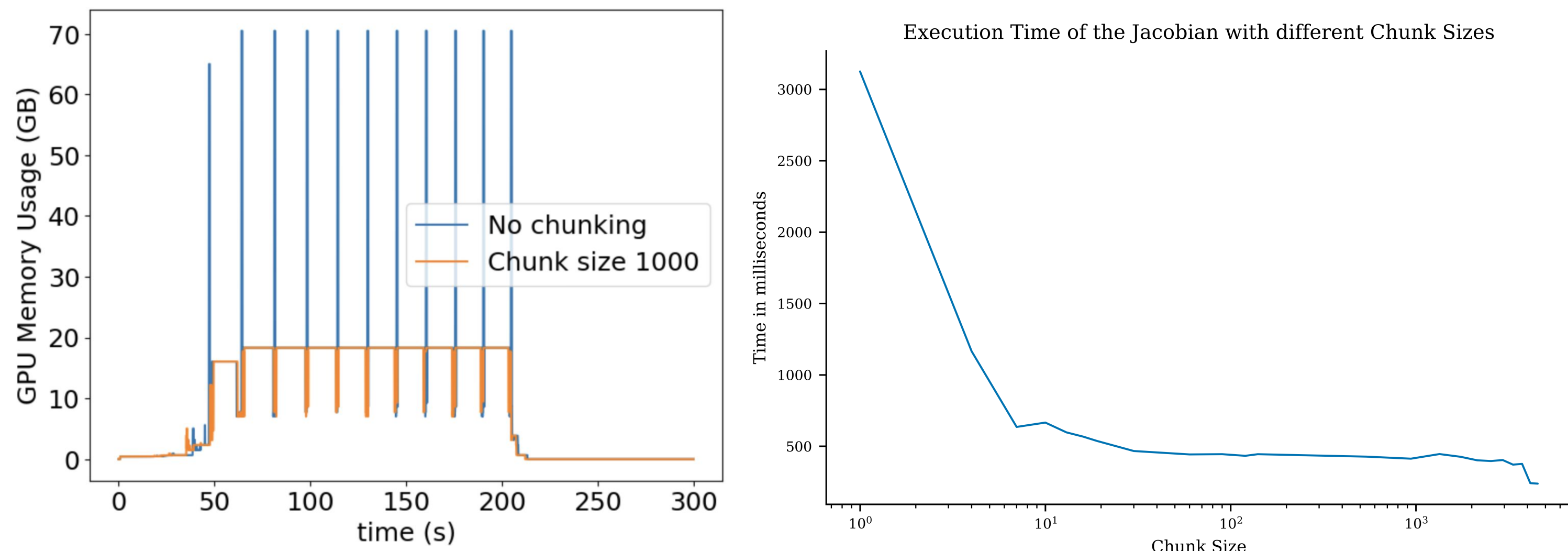


CHUNKING FOR DERIVATIVES

Out of memory (OOM) error have been limiting the optimization problem size in DESC. This was primarily due to the Jacobian calculation. Looped option of Jacobian calculation was a lot slower to be effective. During the NVidia GPU Hackathon last year, we implemented a new way to compute the Jacobian in chunks such that each chunk uses the available memory, and loops over chunks.

$$\begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} & \frac{\partial f_1}{\partial x_4} & \frac{\partial f_1}{\partial x_5} & \frac{\partial f_1}{\partial x_6} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} & \frac{\partial f_2}{\partial x_4} & \frac{\partial f_2}{\partial x_5} & \frac{\partial f_2}{\partial x_6} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \frac{\partial f_m}{\partial x_3} & \frac{\partial f_m}{\partial x_4} & \frac{\partial f_m}{\partial x_5} & \frac{\partial f_m}{\partial x_6} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

chunk size of 2



IMPROVED TRUST REGION

DESC by default uses least-squares with trust region for the optimization. This means for each optimization step, we are solving the trust region subproblem to find the descent direction. The subproblem can be formulated as,

$$\min_{\Delta y} \|f + \nabla f^T \Delta y\|_2^2 + \alpha \|\Delta y\|_2^2$$

$$\text{subject to } \|\Delta y\|_2 \leq r_{tr}$$

Unfortunately, the Jacobian in DESC usually ill-conditioned, therefore we solve this by matrix decomposition methods. Given SVD, we try to find alpha value that gives norm of delta y equal to trust radius.

$$\Delta y = -\mathbf{V}(\Sigma^T \Sigma + \alpha \mathbb{I})^{-1} \Sigma \mathbf{U}^T f$$

One can equivalently use QR decomposition which is faster. By switching to QR, we achieve on average twice a speed up for this subproblem with high resolution problems..

$$\begin{bmatrix} \nabla f^T \\ \sqrt{\alpha} \mathbb{I} \end{bmatrix} \Delta y = - \begin{bmatrix} f \\ 0 \end{bmatrix}$$

AVAILABLE OBJECTIVES in DESC

Quasisymmetry
General Omnigenity
EP confinement (FC, Γa)
Effective Ripple
Magnetic Well
Mercier Stability

Ideal Ballooning Stability
Mean/Principal Curvature
Magnetic gradient scale length (LVB)
Elongation
REGCOIL (Python version, GPU)
Bootstrap Current (for QS via RedI)

Coil Length
Coil-Coil Distance
Coil-Plasma Distance
Quadratic Flux
QFM Surfaces
Coil Curvature

Dommaschk Potentials
Rotational Transform
Shear
Aspect Ratio
Volume
Coil Torsion
Coil Strain

Force Balance
MHD Energy
Anisotropic Force Balance
Free Boundary
NAE-Constrained Equilibria
Fusion Power
Required Heating Power

Nestedness* (Via Laplace Solver)
Monoenergetic Transport Coefficients (via MONKES in JAX)*
Neoclassical Transport (via YANCC, a JAX neoclassical code)*
Energetic Particle orbit tracing*
Maximum-J*
QUADCOIL*
Finite Build Coils*

* In Progress

CURRENT WORK on MPI PARALLELISM

For realistic reactor designs, we need to include many objectives to the optimization which increases the memory. Although DESC is fast, easy to use and multithreaded through JAX, it needs to have multidevice capability to get over the memory problem and speed improvement.

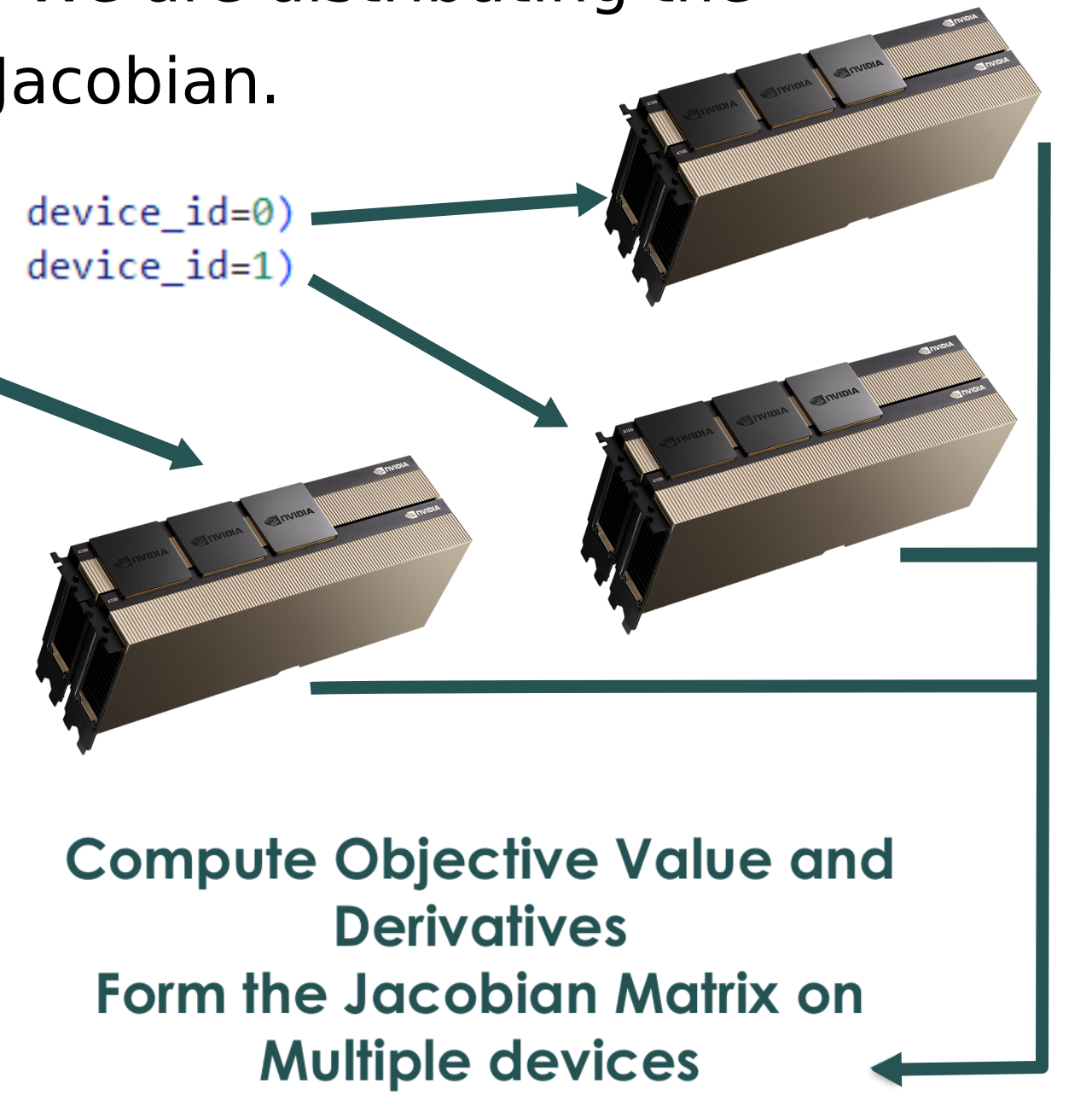
Native JAX only supports single program multiple device (SPMD) parallelism. As SIMSOPT, we built an API with MPI using mpi4py. We are distributing the objectives to different devices and form the full Jacobian.

```
obj1 = QuasisymmetryTwoTerm(eq=eq, helicity=(1, eq.NFP), grid=grid1, device_id=0)
obj2 = QuasisymmetryTwoTerm(eq=eq, helicity=(1, eq.NFP), grid=grid2, device_id=1)
obj3 = AspectRatio(eq=eq, target=8, weight=100, device_id=2)
objs = [obj1, obj2, obj3]
objective = ObjectiveFunction(objs, deriv_mode="blocked", mpi=MPI)
```

Although implemented, to merge in master, we need more extensive testing and feedback from users.

Expected improvements,

New Memory Limit :
most expensive objective
Speed up for cost/Jacobian:
 $\frac{\sum \text{time per objective}}{\text{time for the most expensive objective}}$



TRENDS in OMNIGENITY DATABASE (RAHUL GAUR)

We have a database of ~100k omnigenous equilibria with an aspect ratio $A \leq 10$ generated using DESC.

For OP, access to higher NFPs improves upon increasing beta

